ParkOps: a supervised parking-operations agent

EECS 3311 B · Fall 2026 · Individual project · Three cumulative deliverables

Release	Deadline, Toronto time	Course weight	Focus
D1	October 9, 2026, 23:59	10%	Domain design and executable proposal path
D2	November 6, 2026, 23:59	10%	Working model/tool loop, approval and tests
D3	November 30, 2026, 23:59	15%	Changed policy, refactoring and contracts

The three submissions extend the same system. Preserve earlier tests unless a published change deliberately changes their expected behaviour. These dates and weights are the Section B release baseline; the instructor must complete common-section moderation before publishing a common assessment scheme.

The operational task

A reserved parking space closes. An agent receives the goal "Resolve the booking affected by the closure of A17." It reads the affected booking, obtains candidate spaces, proposes a permitted reassignment, and waits for a human operator. The application executes only an approved proposal after checking the current data again. It records the result and a simulated notification. A repeated execution request must not commit the same proposal twice.

The model decides what permitted information to request and what proposal to make. Java code checks the request, calls the tool, returns an observation, enforces the limit on iterations and protects domain state. A model-generated sentence recommending another space is not the completed workflow. A scripted sequence is a useful test double, not a live AI model.

This is a synthetic, single-process, sequential, in-memory teaching system. The approval operator is represented by a separate local application command, not production authentication. No real parking service, payment, email, SMS, customer record or external account is involved. The project does not establish concurrent database safety, crash recovery, distributed exactly-once delivery, production security certification or clinical/other high-stakes readiness.

Scope and supplied resources

Students implement the domain objects, application workflow, tool dispatch, proposal lifecycle, periodic check, tests and design documentation. The starter supplies a small JSON codec, a `ModelProvider` interface, a deterministic `ReplayProvider`, a bounded HTTP adapter for an Ollama-compatible model endpoint, build scripts and synthetic data. The supplied infrastructure is not the completed project and its two smoke checks do not establish project acceptance. You may improve or replace supplied infrastructure if you preserve the required public behaviour and document the change.

Use Java 21. A command-line interface is sufficient. Do not add model training/fine-tuning, a substantial frontend, production payments, unrestricted shell/browser tools, automated purchases or infrastructure-heavy deployment. Model weights and provider accounts are not deliverables. Keep components small enough to explain and test. There is no minimum design-pattern count.

Required domain rules

Identifiers match `[A-Z][A-Z0-9_-]{0,15}`: an uppercase initial letter followed by at most fifteen uppercase letters, digits, underscores or hyphens. Null, blank, lowercase and overlong identifiers are rejected. Duration

is a whole number of hours from 1 through 24 inclusive. Store money as integer cents where a pricing exercise uses it; no payment is taken.

A booking has an identifier, assigned space identifier, duration, booking version and an accessibility-requirement flag. A space has an identifier and open, occupied and accessible flags. "Available" means open and not occupied. A proposal records its own identifier, booking, target space, observed booking version, observed policy version and status. The statuses are PENDING, APPROVED, REJECTED and EXECUTED. The model never supplies approval.

D1/D2 use **policy version 1**: the target must differ from the current space and be available. The accessibility flag is deliberately present but unused in this simplified baseline; this is a controlled evolution exercise, not an acceptable real parking policy. D3 activates policy version 2, which also requires an accessible target when the booking requires one.

Supplied scenario

Space	Open	Occupied	Accessible
A17	No	Yes	No
B12	Yes	No	No
C03	Yes	No	Yes
D09	Yes	Yes	Yes

Booking B1 starts at A17 for three hours, booking version 0, and requires an accessible space. Under the baseline policy, the eligible alternatives are B12 and C03. Under policy version 2, only C03 is eligible. Do not silently change this fixture to make a test pass. You may use a constructor-based fixture or load the supplied JSON; a database is not required.

Model request and observation protocol

A model response is one JSON object with **exactly** these three string properties:

```
{"tool":"READ_BOOKING","bookingId":"B1","targetId":""}
```

Allowed `tool` values are `READ_BOOKING`, `LIST_ALTERNATIVES`, `PROPOSE_REASSIGNMENT` and `FINISH`. `bookingId` must be the assigned goal booking. `targetId` is empty except for a proposal, where it must be a valid known space identifier. Reject additional properties, missing properties, non-string values, malformed JSON and unknown tool names. In particular, `approved:true` is not accepted as authority. The application supplies proposal identifiers.

READ_BOOKING returns a current booking snapshot and policy version. LIST_ALTERNATIVES requires a preceding read and returns candidates from the current state; reject or restart the investigation if the captured versions are stale. PROPOSE_REASSIGNMENT requires the read and list steps, validates the chosen target and captured versions, creates one pending proposal, and returns AWAITING_APPROVAL. FINISH returns NO_ACTION without changing a booking.

Each actual model invocation counts toward an integer limit from 1 through 6. Invalid responses consume an attempt. A controlled tool-validation error can be returned as an observation and the agent can try again within the limit. Exhaustion returns STEP_LIMIT; a provider exception or timeout returns PROVIDER_ERROR. Neither outcome commits a booking. Treat a response as data, never as executable code, shell commands or unrestricted method names.

Approval, execution and duplicate behaviour

Only a separate operator action can approve or reject a pending proposal. Approval and rejection are legal only from PENDING. Rejected proposals cannot execute. Execution first recognises a previously committed

proposal; its repeated execution returns a duplicate result and does not mutate again. Otherwise it requires APPROVED, matching current booking/policy versions, and a currently eligible target.

Use distinguishable outcomes corresponding to `NOT_APPROVED`, `STALE`, `UNAVAILABLE_OR_INELIGIBLE`, `COMMITTED` and `DUPLICATE`. Exact Java class and method names are your design choice; describe the mapping in the README. A successful commit releases the old occupancy, occupies the target, moves the booking, increments its version once, marks the proposal executed, records the commit and creates one simulated notification record. A rejection must leave booking location, version, occupancy and commit count unchanged.

The sequential boundary must contain validation and mutation without an intervening external/model call. Do not claim that a sequential check-then-update alone is safe under concurrent processes. Persistence across restarts is outside scope. Use immutable snapshots or defensive copies to prevent callers from bypassing controlled mutation.

Periodic checks without waiting in real time

Implement an explicit `tick(time)` with a controllable monotonic time source. Use a 60-second period in the required example. The first tick may scan immediately. A tick before the next boundary returns no new work; the boundary tick scans. A clock value lower than the previous tick is rejected.

An affected booking is one assigned to a closed space. Emit at most one automatic work item for a given booking-version/policy-version pair and suppress one with an active pending/approved proposal for that same pair. Repeated ticks must not create duplicate proposals. Under the supplied baseline, a failed attempt requires an explicit operator retry or a relevant version change; explain this conservative policy. A background daemon, sleep loop or hosted scheduler is unnecessary.

Live-model access and fair testing

The included HTTP adapter is a real model-connection route, not a model. It requires an available model endpoint and a model already installed there. The course must announce a supported endpoint/lab machine and a tested model configuration before the live-run requirement is activated. No student must buy API credits or provide a personal paid account. Use only synthetic data.

The live demonstration must retain the actual model identifier, request/observation transcript, resulting proposal and operator outcome. The model may choose a different eligible target from a replay; grade permitted task behaviour, not identical wording or one preferred eligible identifier. A successful happy-path transcript does not replace failure tests.

All core marking is reproducible with replay fixtures. If the course has not supplied working model access, submit the implemented adapter boundary, replay checks and an explicit `LIVE_RUN_PENDING_COURSE_ACCESS` status. The live demonstration will then be completed in the announced supported session without a penalty for absent access. Do not fabricate a transcript or relabel replay. The instructor must resolve and announce this access arrangement early enough for D2.

Design and marking expectations

Use classes, interfaces and patterns to address actual responsibilities and variation. Diagrams must match your submitted implementation. Explain one alternative rejected for each major decision. Tests need meaningful expected results and unchanged-state checks after rejection. Contracts must state preconditions, postconditions and allowed state changes for selected deterministic operations. Static-analysis results require interpretation, not a screenshot of a tool badge.

The workload is deliberately bounded for one student: planning estimates are 12–16 hours for D1, 18–22 for D2 and 16–20 for D3, in addition to related lab practice. These are instructor planning estimates, not

measured student completion times. Report material setup barriers early. Additional product features are optional and do not compensate for missing required behaviour.

References and provenance

The parking domain and software-design sequence adapt the supplied Song Wang Winter 2023 materials. The supervised-agent workflow and Fall 2026 requirements are newly specified for this course. Java semantics: Oracle Java SE 21 specification. Model protocol infrastructure: Ollama Chat API and structured-output documentation. Agent permissions: OWASP GenAI LLM06:2025 Excessive Agency. Full source URLs are in the syllabus reference appendix.

Submission and individual authorship

This is an individual project. Submit your own repository snapshot and explain every included design decision. Do not submit a shared implementation, team report or another student's source. Discussion of general concepts is permitted; exchange of assessed code, diagrams or solutions is not. Designated AI assistance is permitted with the disclosure below. Supplied infrastructure is identified explicitly and may be retained with attribution.

Submit one ZIP through the named eClass assignment before **23:59 Toronto time** on the stated date. Name it `EECS3311_Dn_STUDENTNUMBER.zip`, replacing `Dn` and `STUDENTNUMBER`. Its single top-level folder must contain `src/`, `test/`, `run.sh`, `README.md`, `DESIGN.pdf`, `AI_USE.md`, `TEST_PLAN.md`, `CHANGELOG.md` and `results/`. Keep generated `.class` files, IDE caches, model weights, secrets and downloaded libraries out. A private Git repository is recommended for development; the submitted ZIP is the marking snapshot. Do not rely on a remote branch changing after the deadline. Keep a copy of the uploaded receipt and ZIP.

`README.md` must state the JDK, exact execution commands, supported scenario(s), known limitations and any permitted starter files reused. Both `bash run.sh check` and the appropriate prepared demonstration must run from the extracted project root. Jupiter tests use `bash setup-junit.sh` once on the supported network, then `bash run.sh test`; the TA will provide/cache the pinned dependency. Include genuine command output with date, command and environment in `results/`. Do not label a plain-Java check run as a JUnit run or label replay as a live-model run.

In `AI_USE.md`, identify each tool/model used, date or version when known, the task it helped with, the source files or diagrams affected, what you changed or rejected, and how you checked the result. State "No generative AI assistance used" when applicable. Whole personal chat histories are not required. Remove credentials and personal information from any excerpt. Code produced with assistance remains your responsibility. The instructor may ask you to explain or make a small change to your submitted work under the course's academic-integrity procedures; there is no separate undisclosed viva mark.

Submit even if one part is incomplete: identify the affected requirement precisely. Do not hide a failure by deleting the test or editing a captured result. Required extensions cannot be replaced by a polished user interface. Optional features earn no substitute marks for missing core requirements. Approved accommodations and extensions take precedence over ordinary deadlines; request assistance through eClass promptly. Late/absence arrangements are governed by the published syllabus and approved course procedures, not by this brief inventing a separate penalty.